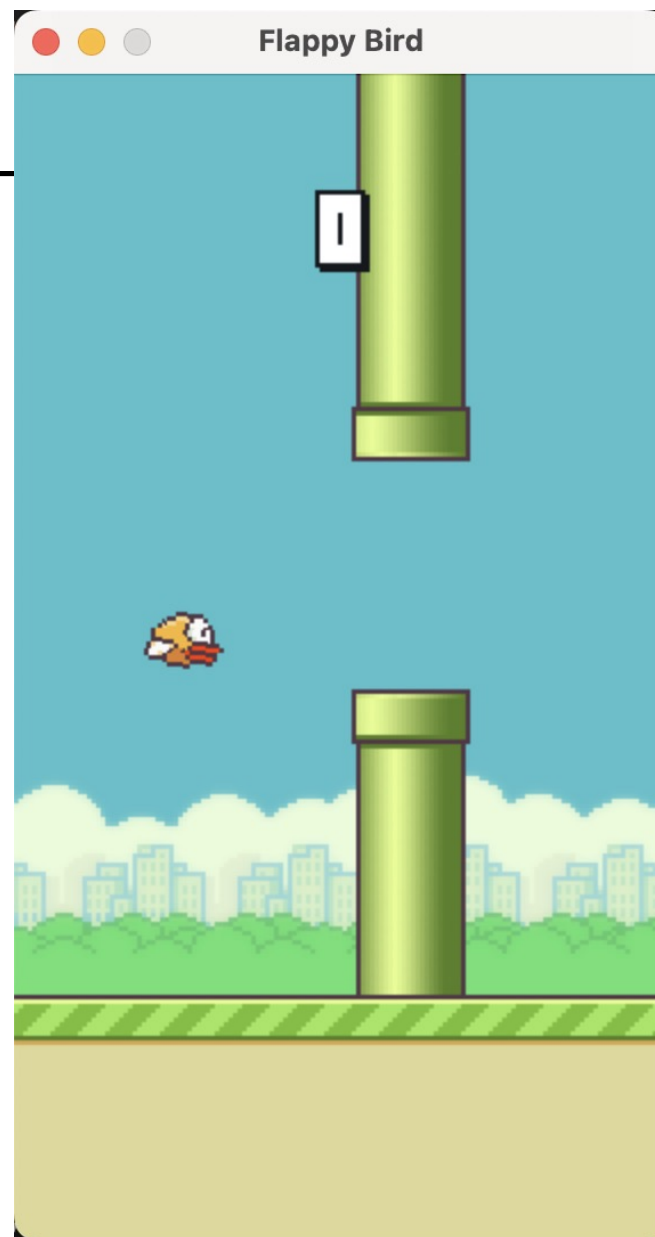


问题描述

Flappy Bird是一款2013年发布的经典2D游戏。游戏简单而富有挑战性，玩家需要控制小鸟穿过上下起伏的障碍物（管道），避免碰撞并尽可能飞得更远。

游戏开始时，小鸟从左侧向右飞行，同时会不断地向下坠落。玩家通过点击按键来使小鸟向上飞行一定高度（表现为小鸟的Flap动作）。每次点击小鸟都会向上飞行一定高度，并在之后由于重力作用逐渐下落。在飞行的过程中，小鸟会遇到从上方和下方伸出的障碍物（管道）。玩家需要控制小鸟穿过管道之间的空隙，避免碰撞到管道，否则游戏结束。每当小鸟穿过一对管道之间的空隙时，玩家会累积一分。

为Flappy Bird这个游戏设计一个智能体，通过强化学习的方式使其能够在这个游戏中获得尽可能高的分数。



目录

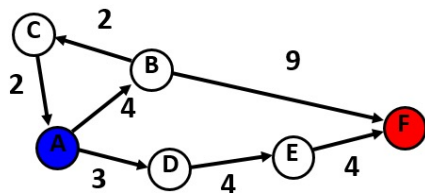
- 马尔科夫决策过程 **Markov Decision Processes (MDP)**
 - 定义
 - 序列效用 (utilities of sequence)
 - 最优量 (Optimal Quantities)
 - **Bellman equation for utilities**
 - **Q-Value**
 - **Value Iteration** 值迭代
 - Policy Iteration 策略迭代
- 强化学习
 - 简介
 - **Q-learning**

MDP 不确定性

确定的环境



随机的环境



state s , action a

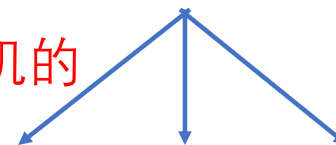


确定的

state Successor(s , a)

state s , action a

随机的



state s_1

state s_2

state s_3

MDP 示例 机器人导航

问题:

- 智能体在格子中
- 墙壁挡住了路

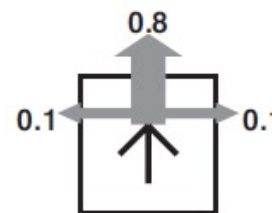
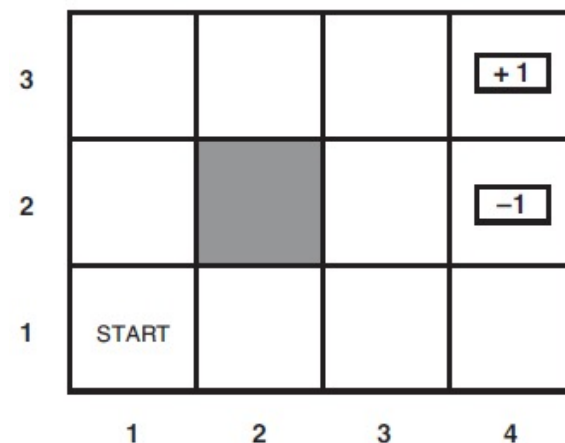
有噪声的环境:

- 80%的情况下, 行动“向北”会向北(如果没有墙)
- 10%的情况下, 会向东; 10%情况下, 会向西
- 如果在agent的移动方向上有一堵墙, 那么agent将停留在同一个方格中。

代理在每个时间步都会收到奖励

- 每一步小小的“生存”奖励(可以是消极的)
- 最后会有大奖励(不管是好是坏)

目标: 最大化效用(奖励总和)



MDP 定义

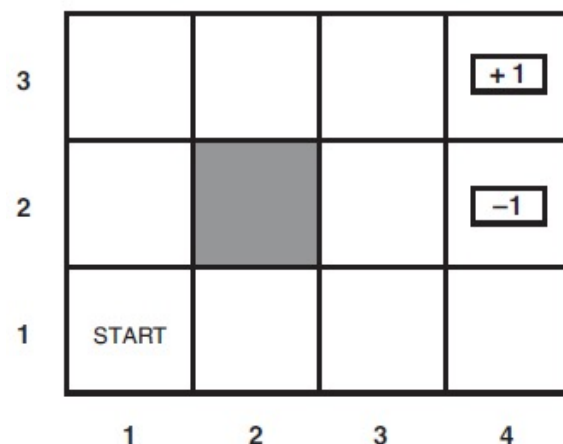
■ 一个马尔可夫决策过程（MDP）由以下要素构成：

- **状态集合**： $s \in S$ （所有可能的状态）
- **动作集合**： $a \in A$ （所有可能的动作）
- **转移函数**： $T(s, a, s')$
 - 表示在状态 s 下执行动作 a 后转移到状态 s' 的概率，即 $P(s'|s, a)$
 - 也称为模型或动态特性
- **奖励函数**： $R(s, a, s')$
 - 有时简化为 $R(s)$ 或 $R(s')$ （仅依赖状态）
 - 初始状态
 - 可能的终止状态

■ 求解目标：

- 策略 π ：为每个状态指定一个动作

■ MDP 是**非确定性的搜索问题**



MDP 特点

- Markov表示，给定现在的状态，过去和未来的状态之间是独立的
- 行动的结果仅和当前状态有关

$$\begin{aligned} &P(S_{t+1} = s' | S_t = s_t, A_t = a_t, S_{t-1} = s_{t-1}, A_{t-1}, \dots, S_0 = s_0) \\ &= P(S_{t+1} = s' | S_t = s_t, A_t = a_t) \end{aligned}$$

序列效用 *utilities of sequence*

- 加性奖励

$$U_h([s_0, s_1, s_2, \dots]) = R(s_0) + R(s_1) + R(s_2) + \dots$$

- 折扣奖励

$$U_h([s_0, s_1, s_2, \dots]) = R(s_0) + \gamma R(s_1) + \gamma^2 R(s_2) + \dots$$

- 为何使用折扣奖励？

- 近期奖励通常比远期奖励更具实际效用（例如，现在获得100元比1年后获得100元更有价值）
- 确保算法收敛

- 例子：折扣因子为0.5

- $U([1,2,3]) = 1*1 + 0.5*2 + 0.25*3$
- $U([1,2,3]) < U([3,2,1])$

- 对于折扣奖励，无限状态序列的总效用始终有限：

$$U_h([s_0, s_1, s_2, \dots]) = \sum_{t=0}^{\infty} \gamma^t R(s_t) \leq \sum_{t=0}^{\infty} \gamma^t R_{\max} = R_{\max} / (1 - \gamma)$$

最优量 *Optimal Quantities*

- 状态s的效用

$$U^\pi(s) = E \left[\sum_{t=0}^{\infty} \gamma^t R(s_t) | \pi, s_0 = s \right]$$

- $U^*(s)$ = 从状态 s 开始并采取最优策略的期望效用。

- 最优策略

$$\pi_s^* = \operatorname{argmax}_{\pi} U^\pi(s)$$

- $\pi^*(s)$: 从状态 s 出发的最优动作。

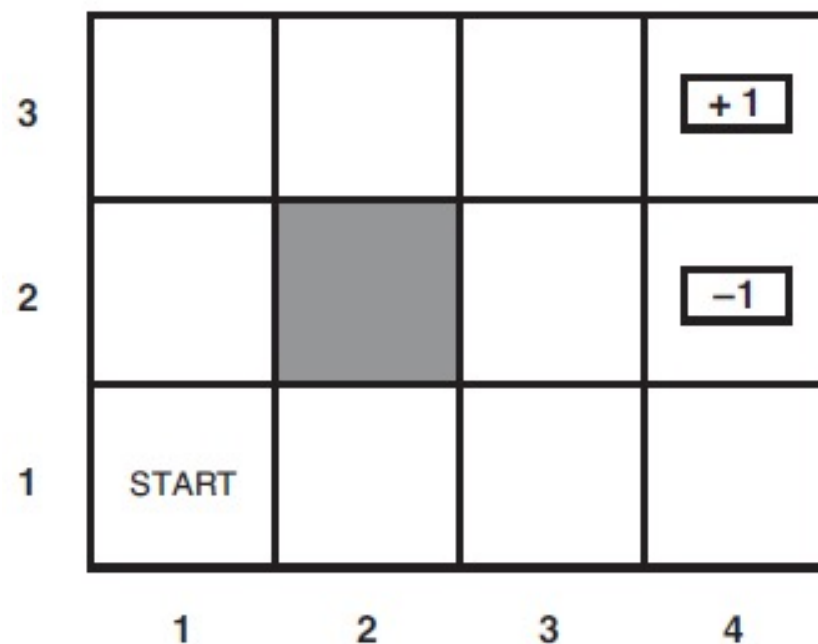
$$\pi^*(s) = \operatorname{argmax}_{a \in A(s)} \sum_{s'} P(s'|s, a) U(s')$$

最优价值决定最优策略

Bellman equation for utilities

$$U(s) = R(s) + \gamma \max_a \sum_{s'} T(s, a, s') U(s')$$

- 贝尔曼方程定义了状态的效用
- 若有 n 个状态，则需解 n 个贝尔曼方程
- 这是一个联立方程组
- 但由于存在 $\max$ 算子，这些方程是非线性的



Bellman equation for utilities

$$U(s) = R(s) + \gamma \max_{a \in A(s)} \sum_{s'} P(s' | s, a) U(s')$$

Noisy 环境.
有10%概率向左.
10%概率向右

$$U(1,1) = -0.04 + \gamma \max \left[\begin{array}{ll} 0.8U(1,2) + 0.1U(2,1) + 0.1U(1,1), & (Up) \\ 0.9U(1,1) + 0.1U(1,2), & (Left) \\ 0.9U(1,1) + 0.1U(2,1), & (Down) \\ 0.8U(2,1) + 0.1U(1,2) + 0.1U(1,1) \end{array} \right]. \quad (Right)$$

3	0.812	0.868	0.918	+1
2	0.762		0.660	-1
1	0.705	0.655	0.611	0.388

1 2 3 4
 $\gamma = 1$ and $R(s) = -0.04$ for nonterminal states.

Q-Value

效用函数

$$U(s) = R(s) + \gamma \max_{a \in A(s)} \sum_{s'} P(s' | s, a) U(s')$$

Q函数

$$U(s) = \max_a Q(s, a)$$

最优策略

$$\pi^*(s) = \operatorname{argmax}_a Q(s, a)$$

$$\begin{aligned} Q(s, a) &= \sum_{s'} P(s' | s, a) [R(s, a, s') + \gamma U(s')] \\ &= \sum_{s'} P(s' | s, a) [R(s, a, s') + \gamma \max_{a'} Q(s', a')] \end{aligned}$$

function Q-VALUE(*mdp*, *s*, *a*, *U*) **returns** 一个效用值

$$\mathbf{return} \sum_{s'} P(s' | s, a) [R(s, a, s') + \gamma U[s']]$$

值迭代

function VALUE-ITERATION(mdp, ε) **returns** 一个效用函数

inputs: mdp , 具有状态 S 、动作 $A(s)$ 、转移模型 $P(s' | s, a)$ 、奖励 $R(s, a, s')$ 、折扣 γ 的MDP
 ε , 任意状态效用允许的最大误差

local variables: U, U' , S 中状态的效用向量, 初始为0
 δ , 任何状态的效用的最大相对变化

repeat

$U \leftarrow U'; \delta \leftarrow 0$

for each 状态 s **in** S **do**

$U'[s] \leftarrow \max_{a \in A(s)} Q\text{-VALUE}(mdp, s, a, U)$

if $|U'[s] - U[s]| > \delta$ **then** $\delta \leftarrow |U'[s] - U[s]|$

until $\delta \leq \varepsilon (1 - \gamma) / \gamma$

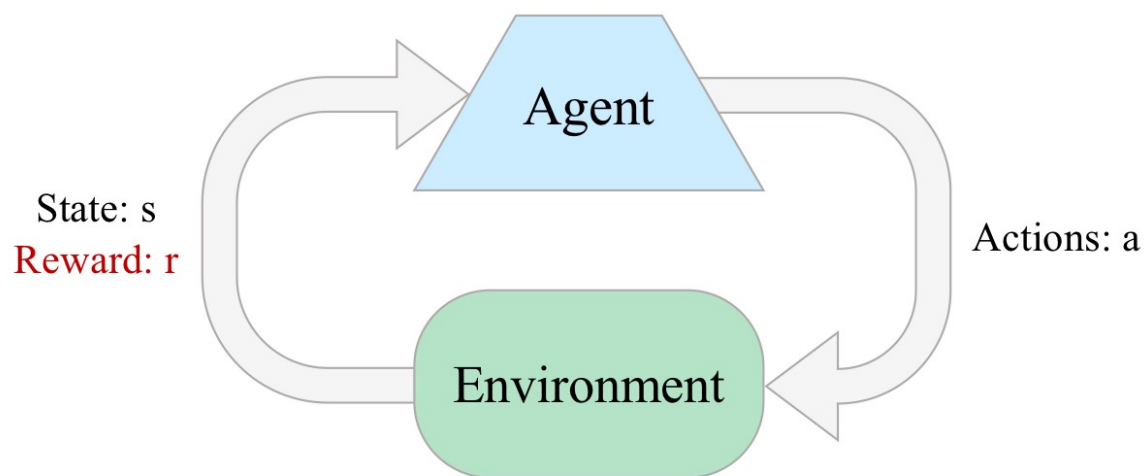
return U

图 17-6 计算状态效用的价值迭代算法。终止条件来自式 (17-12)

强化学习

强化学习 (Reinforcement Learning, RL) 是机器学习的一个分支, 其核心思想是智能体 (Agent) 通过与环境的交互学习最优策略, 以最大化长期累积奖励。

- **智能体 (Agent)** : 学习的决策主体。
- **环境 (Environment)** : 智能体交互的外部世界。
- **状态 (State)** : 环境在某一时刻的描述 (如游戏画面、机器人传感器数据)
- **动作 (Action)** : 智能体在某个状态下可以采取的行为。
- **奖励 (Reward)** : 环境对智能体动作的即时反馈 (正奖励鼓励, 负奖励惩罚)。
- **策略 (Policy)** : 智能体从状态到动作的映射规则 (即如何做决策)。



Q-learning

Q-Learning是一种**基于样本**的Q-Value迭代方法，它是一种**无模型**的强化学习方法，我们只需要从真实世界中采样足够多的四元组样本 (s, a, s', r) （当前状态，行动，新状态，回报），便能较好地估计每一个（状态，行动）对的Q-Value。接下来我们使用 s 表示当前状态， a 表示作出的行动， s' 表示在状态 s 执行行动 a 后转移到的新状态， r 表示在这一过程中获得的回报，Q-Learning 算法的具体步骤如下：

(1) 对于所有的 (s, a) 对，初始化 Q-Value 为0

(2) 从真实世界采样，获得一个样本 $(s, a, s',$

$r)$

$$Q(s, a) = (1 - \alpha)Q(s, a) + \alpha[r + \gamma \max_{a'} Q(s', a')] \quad (1)$$

(4) 回到步骤 (2) 直到Q-Value收敛

Flappy Bird

■ 问题描述

■ 代码框架

- 问题表示 (AI)
- 训练train
- 测试play

■ 代码实现

- 问题表示
 - 状态 (state), 行动 (action), 奖励 (reward)
- 算法实现
 - 训练流程
 - 选择一个行动
 - 获取这个行动带来的reward
 - 更新Q值
 - 测试流程
 - 查询Q值表, 查询当前状态 (state) 下具有最大Q值的行动 (action)
 - 更新状态

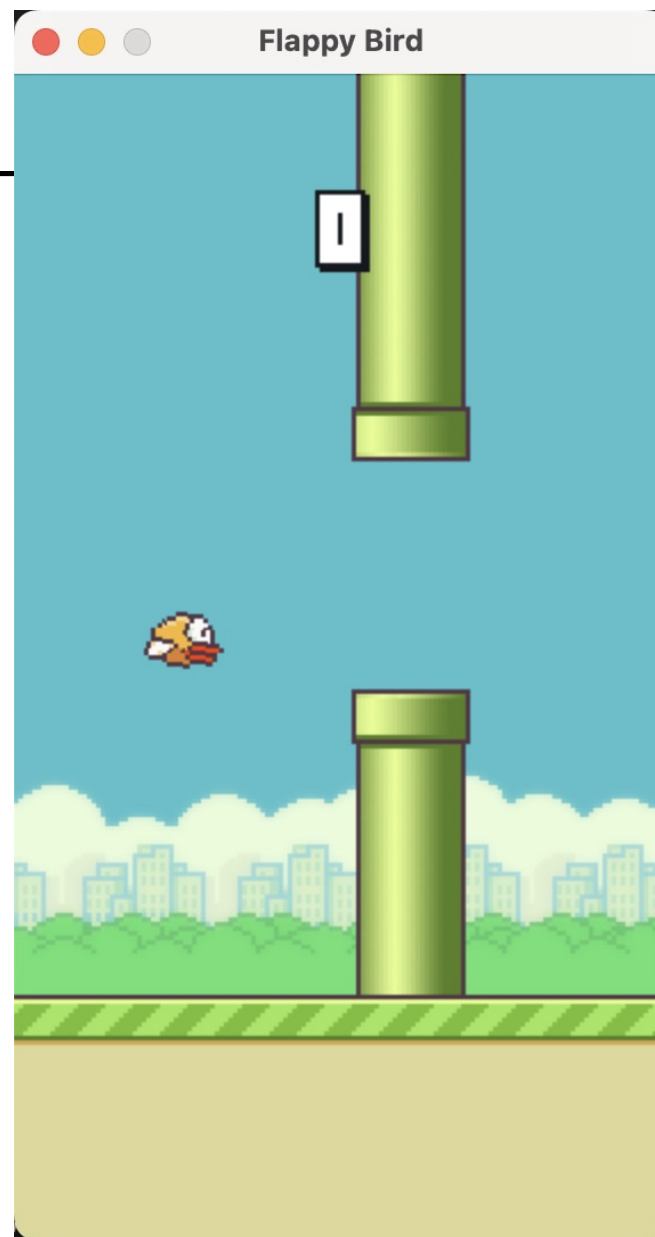
问题描述

Flappy Bird是一款2013年发布的经典2D游戏。游戏简单而富有挑战性，玩家需要控制小鸟穿过上下起伏的障碍物（管道），避免碰撞并尽可能飞得更远。

游戏开始时，小鸟从左侧向右飞行，同时会不断地向下坠落。玩家通过点击按键来使小鸟向上飞行一定高度（表现为小鸟的Flap动作）。每次点击小鸟都会向上飞行一定高度，并在之后由于重力作用逐渐下落。在飞行的过程中，小鸟会遇到从上方和下方伸出的障碍物（管道）。玩家需要控制小鸟穿过管道之间的空隙，避免碰撞到管道，否则游戏结束。每当小鸟穿过一对管道之间的空隙时，玩家会累积一分。

为Flappy Bird这个游戏设计一个智能体，通过强化的学习方式使其能够在这个游戏中获得尽可能高的分数。

(s, a, s', r) （当前状态，行动，新状态，回报）



Flappy Bird

■ 问题描述

■ 代码框架

- 问题表示 (AI)
- 训练train
- 测试play

■ 代码实现

- 问题表示
 - 状态 (state), 行动 (action), 奖励 (reward)
- 算法实现
 - 训练流程
 - 选择一个行动
 - 获取这个行动带来的reward
 - 更新Q值
 - 测试流程
 - 查询Q值表, 查询当前状态 (state) 下具有最大Q值的行动 (action)
 - 更新状态

代码框架-问题表示

```
class GameAI():  
  
    def __init__(self, alpha=0.5, gamma=1, epsilon=0.1):  
        """
```

初始化:

一个字典self.q表示Q-Function, 存储从 (状态, 行动) 对到Q-Value的映射,

Args:

* alpha: 学习率

* gamma: 折扣因子

* epsilon: 行动时的探索概率

"""

```
self.q = dict()
```

```
self.alpha = alpha
```

```
self.gamma = gamma
```

```
self.epsilon = epsilon
```

代码框架-训练测试

```
if args.train:
    now_time = datetime.datetime.now().strftime('%m%d_%H%M%S')
    # 根据当前时间给Q-Function命名
    path = 'q_' + now_time + '.pkl'
    start_time = time.time()
    ai = train(iteration, alpha, gamma, epsilon) 通过强化学习训练一个AI
    interval = int(time.time() - start_time) # Get elapsed time in seconds
    minute = interval // 60
    second = interval % 60
    print("Training time:", f"{minute} min and {second} sec")
    ai.save_q(path)
else:
    ai = GameAI()
    """
    请将'q_timestamp.pkl'替换成你自己的Q-Function的保存路径
    """
    path = 'q_0409_225409.pkl'
    ai.load_q(path)
    play(ai) 根据训练的AI进行游戏测试
```

代码框架-训练

```
def train(iteration, alpha, gamma, epsilon):  
    """  
    通过让AI进行n次游戏来进行强化学习。  
  
    Args:  
    * iteration: 训练时进行的游戏次数  
    * alpha: 学习率  
    * gamma: 折扣因子  
    * epsilon: 行动时的探索概率  
    """  
    player = GameAI(alpha=alpha, gamma=gamma, epsilon=epsilon)  
  
    env = gymnasium.make("FlappyBird-v0", render_mode=None, use_lidar=False)  
    # 使用seed可以确保每次训练时游戏的随机性都是一致的  
    obs, _ = env.reset(seed=42)  
    # 进行多次游戏  
    for i in range(iteration): ...  
  
    env.close()  
    print("Done training")  
    # 返回训练完毕的AI  
    return player
```

代码框架-测试

```
def play(ai, audio_on=False, render_mode="human", use_lidar=False):
    env = gymnasium.make("FlappyBird-v0", audio_on=audio_on, render_mode=render_mode, use_lidar=use_lidar)
    scores = []
    # 同样, 使用seed可以确保每次游戏的随机性都是一致的
    obs, _ = env.reset(seed=42)

    while True:
        action = ai.choose_action(process_obs(obs), use_epsilon=False)

        # Processing:
        obs, _, done, _, info = env.step(action)
        """
        这里将Obs的输出注释掉了, 如有调试需要, 可以自行开启
        """
        # print(f"Obs: {obs}\n" f"Score: {info['score']}\n")

        if done:
            scores.append(info['score'])
            print(f"This try gets {info['score']} score(s).")
            break

    env.close()
    print(f'The average score(s) of Q-Function: {sum(scores) / len(scores)}')
    assert obs.shape == env.observation_space.shape
```

Flappy Bird

■ 问题描述

■ 代码框架

- 问题表示 (AI)
- 训练 `train ()`
- 测试 `play ()`

■ 代码实现

- 问题表示
 - 状态 (state), 行动 (action), 奖励 (reward)
- 代码实现
 - 训练流程
 - 选择一个行动
 - 获取这个行动带来的reward
 - 更新Q值
 - 实验优化
 - 测试流程
 - 查询Q值表, 查询当前状态 (state) 下具有最大Q值的行动 (action)
 - 更新状态

代码实现-问题表示

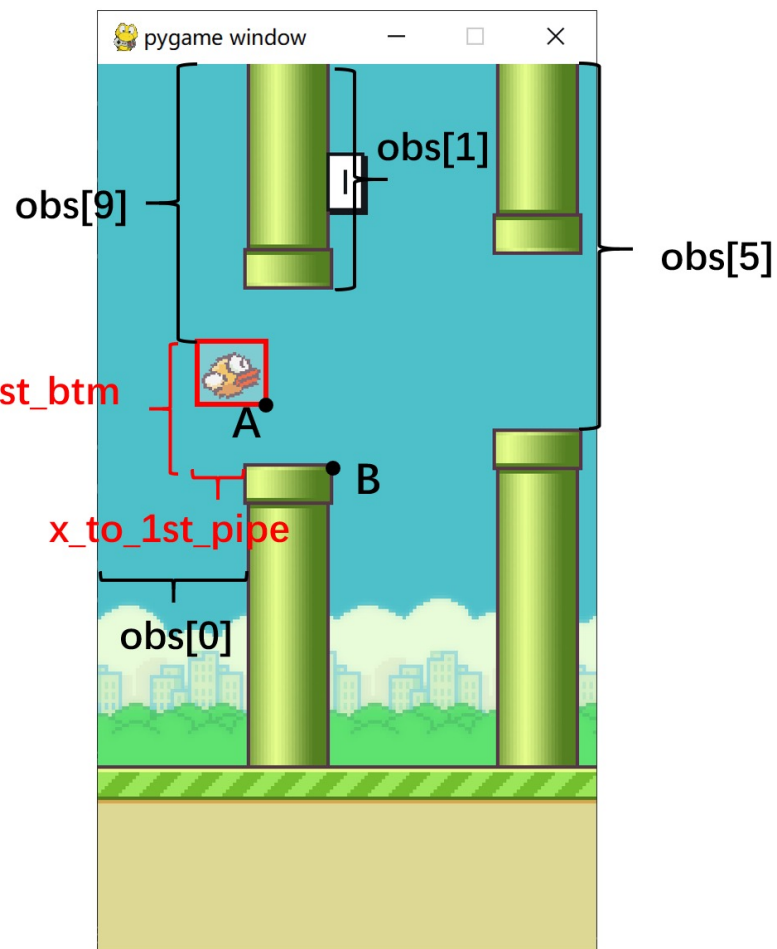
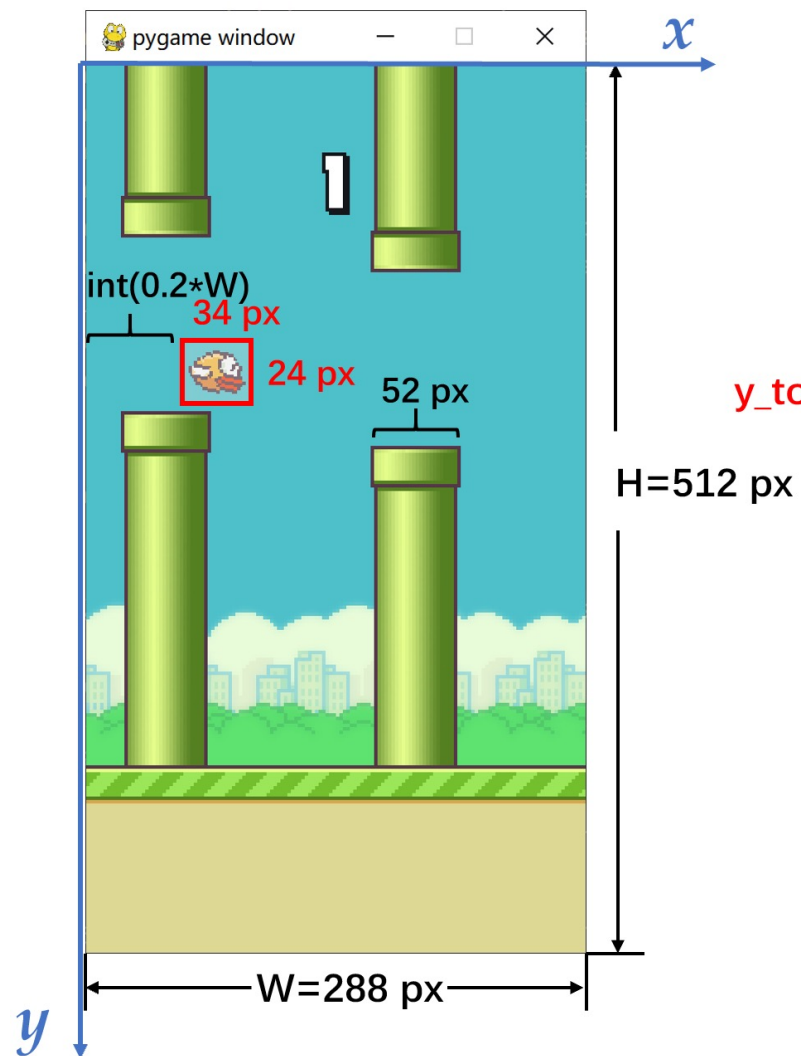
```
class GameAI():  
  
    def __init__(self, alpha=0.5, gamma=1, epsilon=0.1):  
        """  
        初始化:  
        一个字典self.q表示Q-Function, 存储从(状态, 行动)对到Q-Value的映射.  
  
        Args:  
        * alpha: 学习率  
        * gamma: 折扣因子  
        * epsilon: 行动时的探索概率  
        """  
  
        self.q = dict()  
        self.alpha = alpha  
        self.gamma = gamma  
        self.epsilon = epsilon
```

状态（state），行动（action），奖励（reward）

action	值
do nothing	0
flap	1

(s,a)	reward
保持存活	0.1
通过管道	1
触碰天花板	-0.5
触碰地板/管道	-1

代码实现-状态表示



代码实现-状态表示

```
def process_obs(obs) -> List[int]:
```

```
    """
```

通过obs_mul_factor, 将游戏环境返回的各种观测值obs转换成合适的状态值。

obs[0]: the last pipe's horizontal position
obs[1]: the last top pipe's vertical position
obs[2]: the last bottom pipe's vertical position
obs[3]: the next pipe's horizontal position
obs[4]: the next top pipe's vertical position
obs[5]: the next bottom pipe's vertical position
obs[6]: the next next pipe's horizontal position
obs[7]: the next next top pipe's vertical position
obs[8]: the next next bottom pipe's vertical position
obs[9]: player's vertical position
obs[10]: player's vertical velocity
obs[11]: player's rotation

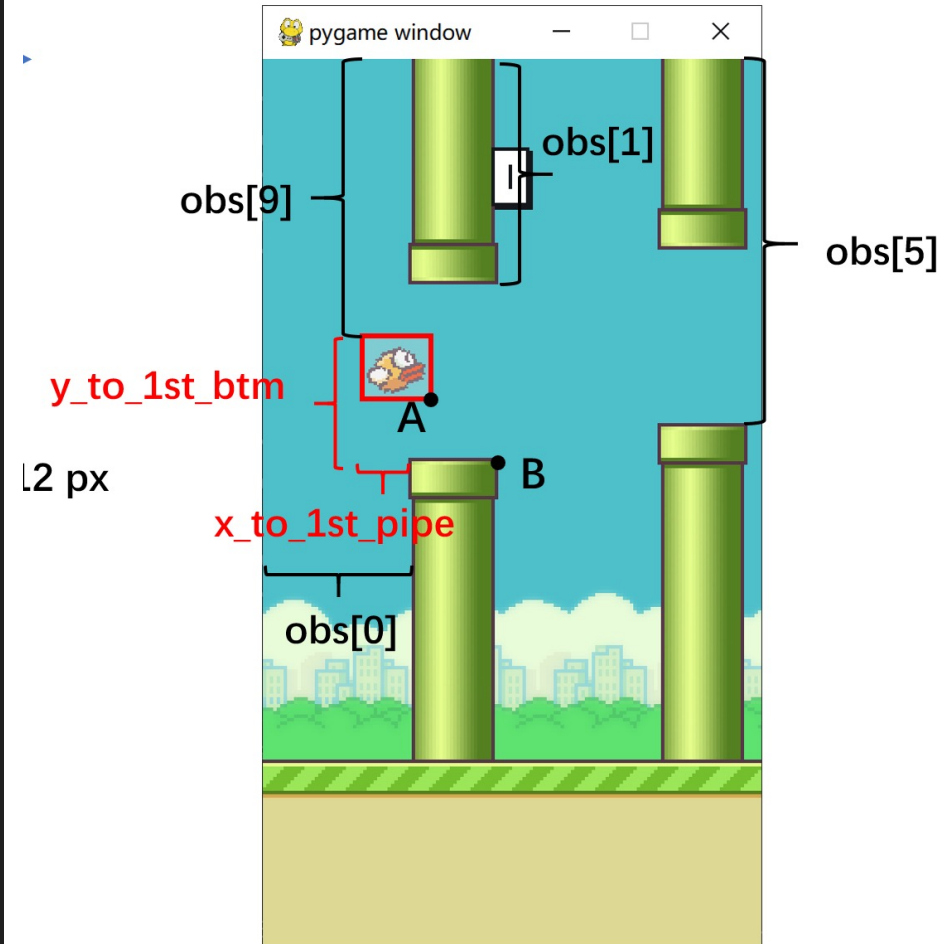
Args:

* obs: Flappy Bird游戏环境返回的各种观测值

Return:

* 根据状态设计, 将当前的观测值转换成对应的状态state

```
    """
```



代码实现-主要函数

```
class GameAI():
```

```
    def __init__(self, alpha=0.5, gamma=1, epsilon=0.1):
```

```
        """
```

```
        初始化:
```

```
        一个字典self.q表示Q-Function, 存储从(状态, 行动)对到Q-Value的映射,
```

```
        Args:
```

```
        * alpha: 学习率
```

```
        * gamma: 折扣因子
```

```
        * epsilon: 行动时的探索概率
```

```
        """
```

```
        self.q = dict()
```

```
        self.alpha = alpha
```

```
        self.gamma = gamma
```

```
        self.epsilon = epsilon
```

```
    def get_q_value(self, state:List[int], action:int) -> float: ...
```

获取当前 (state , action) 对的Q值

```
    def best_future_reward(self, state:List[int]) -> float: ...
```

获取当前state下, 最大的Q值 $\max\{Q(\text{state}, \text{action})\}$

```
    def update(self, old_state:List[int], action:int, new_state:List[int], reward): ...
```

根据Q-leaning更新Q值

```
    def choose_action(self, state:List[int], use_epsilon=True) -> int: ...
```

选择Q值最大的action

```
    @classmethod
```

```
    def available_actions(cls, state:List[int]) -> Set[int]: ...
```

代码实现-训练

```
for i in range(iteration):
    obs, _ = env.reset()
    while True:
        # Next action:
        # (feed the observation to your agent here)
        action = player.choose_action(process_obs(obs))
        # print(action)

        # Processing:
        next_obs, reward, terminated, _, info = env.step(action)
        if reward == -1:
            reward = -1000

        # update the agent
        player.update(process_obs(obs), action, process_obs(next_obs), reward)

        # Checking if the player is still alive
        if terminated:
            break

    obs = next_obs
```

- (1) 对于所有的(s, a)对, 初始化 Q-Value为0
- (2) 从真实世界采样, 获得一个样本 (s, a, s', r)
- (3) 更新 Q-Value:

$$Q(s, a) = (1 - \alpha)Q(s, a) + \alpha[r + \gamma \max_{a'} Q(s', a')]$$

- (4) 回到步骤 (2) 直到Q-Value收敛

实验优化-超参数调整

```
class GameAI():  
  
    def __init__(self, alpha=0.5, gamma=1, epsilon=0.1):  
        """  
        初始化:  
        一个字典self.q表示Q-Function, 存储从(状态, 行动)对到Q-Value的映射,  
  
        Args:  
        * alpha: 学习率  
        * gamma: 折扣因子  
        * epsilon: 行动时的探索概率  
        """  
  
        self.q = dict()  
        self.alpha = alpha  
        self.gamma = gamma  
        self.epsilon = epsilon
```

超参数	含义
alpha	学习率
gamma	折扣因子
epsilon	行动时的探索概率

$$Q(s, a) = (1 - \alpha)Q(s, a) + \alpha[r + \gamma \max_{a'} Q(s', a')]$$

实验优化-Reward调整

```
for i in range(iteration):
    obs, _ = env.reset()
    while True:
        # Next action:
        # (feed the observation to your agent here)
        action = player.choose_action(process_obs(obs))
        # print(action)

        # Processing:
        next_obs, reward, terminated, _, info = env.step(action)
        if reward == -1:
            reward = -1000

        # update the agent
        player.update(process_obs(obs), action, process_obs(next_obs), reward)

        # Checking if the player is still alive
        if terminated:
            break

    obs = next_obs
```

(s,a)	reward
保持存活	0.1
通过管道	1
触碰天花板	-0.5
触碰地板/管道	-1

实验优化-状态表示

```
def process_obs(obs) -> List[int]:
```

```
    """
```

通过obs_mul_factor, 将游戏环境返回的各种观测值obs转换成合适的状态值。

obs[0]: the last pipe's horizontal position
obs[1]: the last top pipe's vertical position
obs[2]: the last bottom pipe's vertical position
obs[3]: the next pipe's horizontal position
obs[4]: the next top pipe's vertical position
obs[5]: the next bottom pipe's vertical position
obs[6]: the next next pipe's horizontal position
obs[7]: the next next top pipe's vertical position
obs[8]: the next next bottom pipe's vertical position
obs[9]: player's vertical position
obs[10]: player's vertical velocity
obs[11]: player's rotation

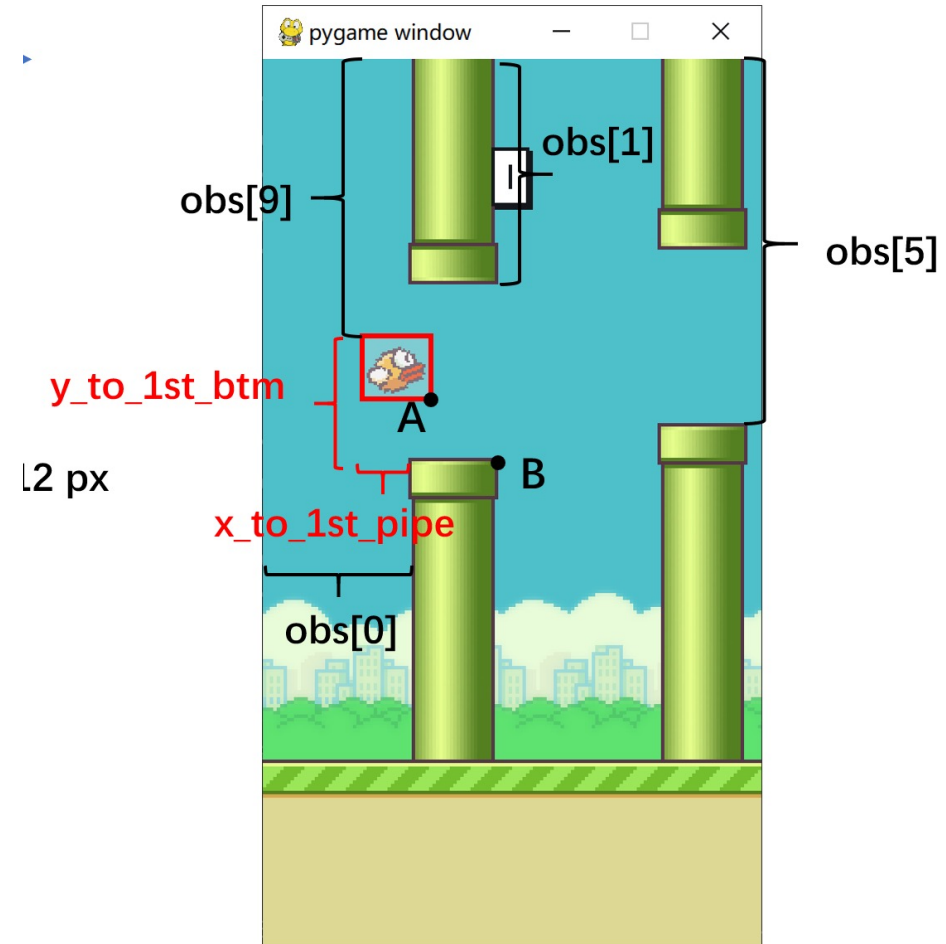
Args:

* obs: Flappy Bird游戏环境返回的各种观测值

Return:

* 根据状态设计, 将当前的观测值转换成对应的状态state

```
    """
```



更合适的表达方式

代码实现-测试流程

```
while True:

    action = ai.choose_action(process_obs(obs), use_epsilon=False)

    # Processing:
    obs, _, done, _, info = env.step(action)
    """
    这里将Obs的输出注释掉了, 如有调试需要, 可以自行开启
    """

    # print(f"Obs: {obs}\n" f"Score: {info['score']}\n")

    if done:
        scores.append(info['score'])
        print(f"This try gets {info['score']} score(s).")
        break
```

作业内容

1. 代码补充(必做): 在src/q_learning.py中, 已经写好了GameAI()类的代码框架, 请你根据TODO1-5的提示和相关的函数说明实现各个部分的代码。
2. 状态表示优化(bonus1): 对于process_obs()这个极为重要的观测值处理函数, 你可以尝试自己设计一个更合理的状态表示, 使得GameAI()在相同的训练参数下能有更好的表现。
3. 训练参数调整(必做): 可以通过适当调整alpha, gamma, epsilon, iteration, mul_obs_factor等参数的大小, 比较一下不同参数设置下的模型表现。
4. reward调整(bonus2): 可以尝试调整游戏环境在不同情况下返回的reward, 比较一下模型表现是否有预期的变化。
5. 选择一个输出平均分数最高的模型, 将其重命名为q_best.pkl进行提交。